

Importing Libraries

```
In [ ]: #download and importing libraries
!pip install numpy pandas matplotlib
!pip install --upgrade pyzmq
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

A.1. Practicing with Numpy arrays

A.1 ⇒ a Printing the type of arr object

```
In [3]: #1D matrix
d1 = np.array([1, 2, 3])
#2D matrix
d2 = np.array([[1,2,3],
               [4,5,6]])
#3D matrix
d3 = np.array([[[1,2,3],
                 [4,5,6]],
               [[7,8,9],
                 [10,11,12]]])

print("1D Matrix:\n {}".format(d1))
print("2D Matrix:\n {}".format(d2))
print("3D Matrix:\n {}".format(d3))
```

1D Matrix:
[1 2 3]

2D Matrix:
[[1 2 3]
[4 5 6]]

3D Matrix:
[[[1 2 3]
[4 5 6]]

[[7 8 9]
[10 11 12]]]

A.1 ⇒ b) Printing array dimensions

A.1 ⇒ c) Printing the shape of the array

A.1 ⇒ d) Printing size (total number of elements) of the array

A.1 ⇒ e) Printing the type of elements in an array

```
In [20]: def number_of_elements(t):
    res = 1
    for i in t:
        res *= i
    return res

def types(d):
    types = []
    f = np.ndarray.flatten(d)
    for i in f:
        t = list(str(type(i)).split(' '))
        types.append(t[1][1:len(t[1])-2])
    return types

print("d1 array has shape {} and dimentions {} and the total number of element
print("d2 array has shape {} and dimentions {} and the total number of element
print("d3 array has shape {} and dimentions {} and the total number of element
print("array d1 has elements of of types {}".format(types(d1)))
print("array d2 has elements of of types {}".format(types(d2)))
print("array d3 has elements of of types {}".format(types(d3)))
```

d1 array has shape (3,) and dimentions 1 and the total number of elements 3
d2 array has shape (2, 3) and dimentions 2 and the total number of elements 6
d3 array has shape (2, 2, 3) and dimentions 3 and the total number of elements 12
array d1 has elements of of types ['numpy.int64', 'numpy.int64', 'numpy.int64']
array d2 has elements of of types ['numpy.int64', 'numpy.int64', 'numpy.int64',
'numpy.int64', 'numpy.int64', 'numpy.int64']
array d3 has elements of of types ['numpy.int64', 'numpy.int64', 'numpy.int64',
'numpy.int64', 'numpy.int64', 'numpy.int64', 'numpy.int64', 'numpy.int64', 'num
py.int64', 'numpy.int64', 'numpy.int64', 'numpy.int64']

A.1 ⇒ f) Slicing of a given array

```
In [5]: n1 = np.array([1,2,3,4,5,6,7,8,9,10])
n2 = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])

# 1st 7 elements
print("1st 7 elements of the array {} is {}".format(n1,n1[:7]))

# last 7 elements
print("last 7 elements of the array {} is {}".format(n1,n1[2:]))

# middle 6 elements
print("last 7 elements of the array {} is {}".format(n1,n1[2:8]))

#Slice from the index 3 from the end to index 1 from the end
print("from end index 3 to 1 from array {} is {}".format(n1,n1[-3:-1]))

#every even elements
print("all even elements of array {} is {}".format(n1,n1[1::2]))

#every odd elements
print("all odd elements of array {} is {}".format(n1,n1[0::2]))
```

```
#from 2nd element index 1 to 4
print("from array \n {} the 2nd element's index 1 to 4 is {}".format(n2,n2[1,
```

```
1st 7 elements of the array [ 1  2  3  4  5  6  7  8  9 10] is [1 2 3 4 5 6 7]
last 7 elements of the array [ 1  2  3  4  5  6  7  8  9 10] is [ 3  4  5  6  7
8  9 10]
last 7 elements of the array [ 1  2  3  4  5  6  7  8  9 10] is [3 4 5 6 7 8]
from end index 3 to 1 from array [ 1  2  3  4  5  6  7  8  9 10] is [8 9]
all even elements of array [ 1  2  3  4  5  6  7  8  9 10] is [ 2  4  6  8 10]
all odd elements of array [ 1  2  3  4  5  6  7  8  9 10] is [1 3 5 7 9]
from array
[[ 1  2  3  4  5]
 [ 6  7  8  9 10]] the 2nd element's index 1 to 4 is [7 8 9]
```

A.2. Write a program to create a data frame using Pandas

```
In [21]: data = {
    "Student_ID": list(range(1, 31)),
    "Maths":      [92, 45, 78, 65, 88, 34, 72, 56, 81, 42,
                  95, 38, 69, 74, 87, 51, 63, 89, 29, 76,
                  58, 67, 93, 48, 71, 85, 36, 62, 90, 54],
    "Science":    [88, 52, 76, 68, 91, 41, 73, 59, 84, 47,
                  94, 35, 70, 77, 89, 55, 64, 86, 33, 79,
                  61, 72, 96, 50, 75, 87, 39, 66, 92, 58],
    "English":    [85, 48, 73, 62, 90, 38, 70, 54, 82, 44,
                  93, 32, 67, 75, 88, 50, 61, 87, 28, 76,
                  56, 69, 95, 46, 72, 86, 35, 64, 91, 52]
}

df = pd.DataFrame(data)
```

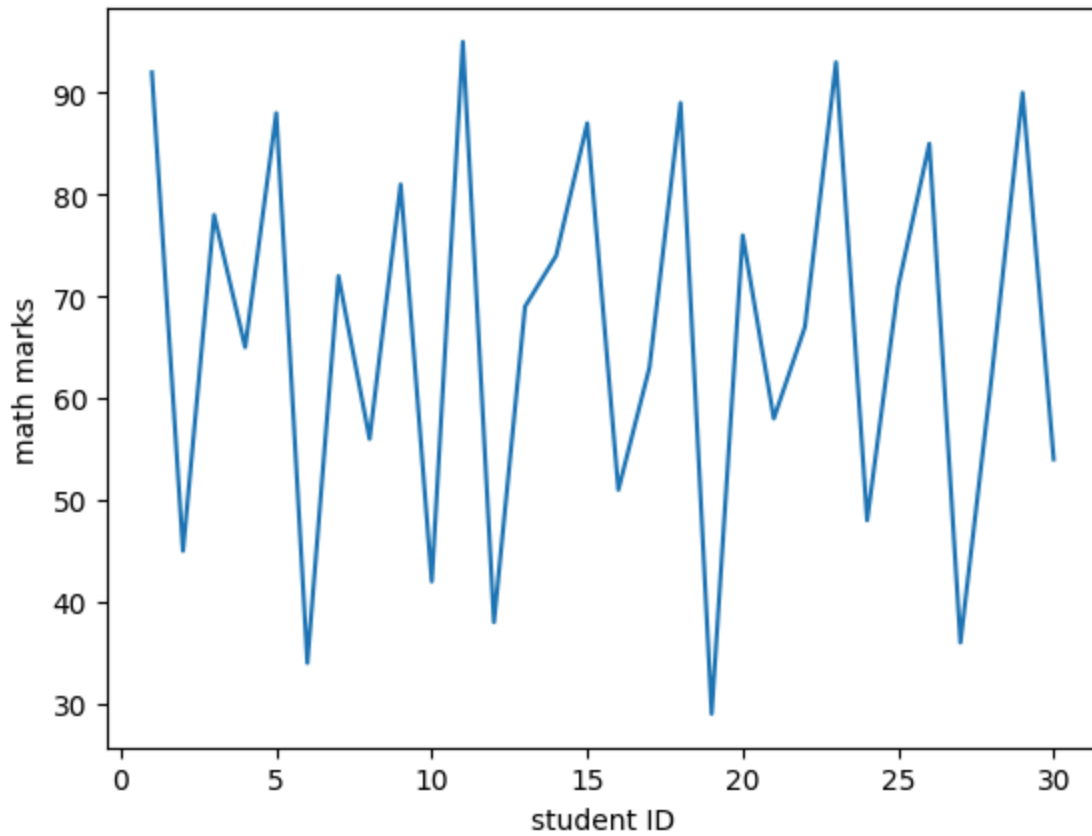
```
In [22]: df
```

Out[22]:

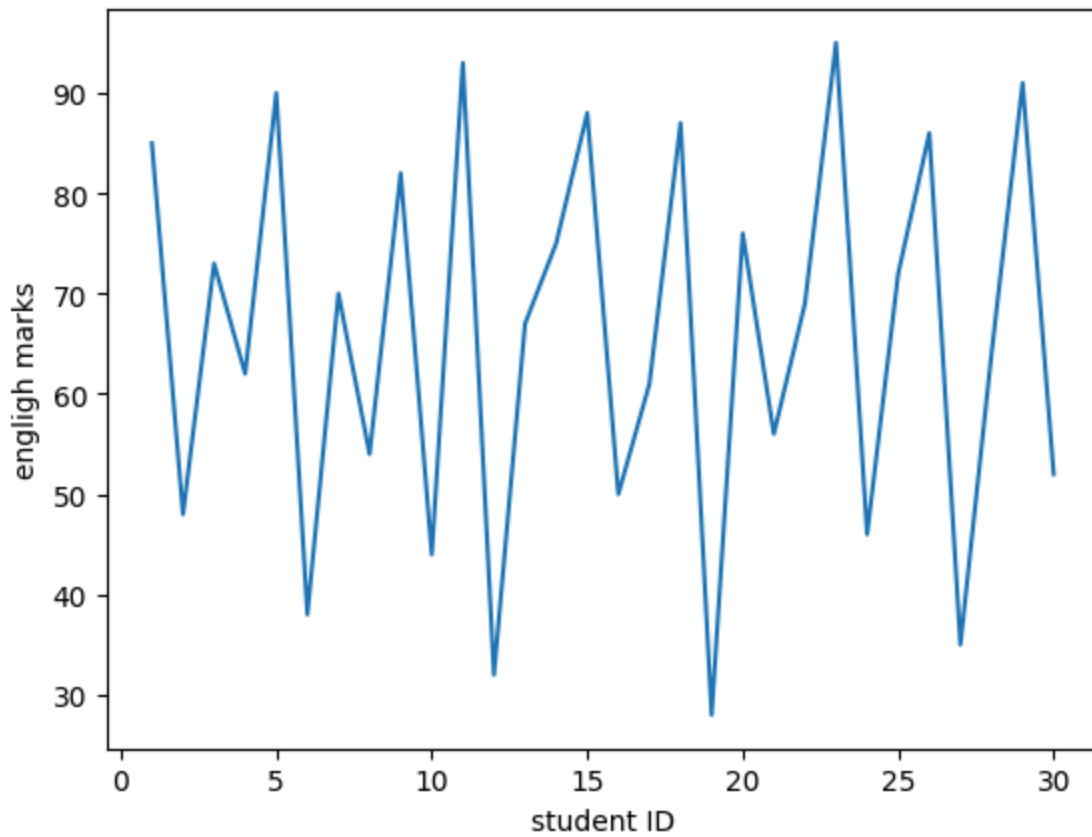
	Student_ID	Maths	Science	English
0	1	92	88	85
1	2	45	52	48
2	3	78	76	73
3	4	65	68	62
4	5	88	91	90
5	6	34	41	38
6	7	72	73	70
7	8	56	59	54
8	9	81	84	82
9	10	42	47	44
10	11	95	94	93
11	12	38	35	32
12	13	69	70	67
13	14	74	77	75
14	15	87	89	88
15	16	51	55	50
16	17	63	64	61
17	18	89	86	87
18	19	29	33	28
19	20	76	79	76
20	21	58	61	56
21	22	67	72	69
22	23	93	96	95
23	24	48	50	46
24	25	71	75	72
25	26	85	87	86
26	27	36	39	35
27	28	62	66	64
28	29	90	92	91
29	30	54	58	52

A.3. Write a Python program for Basic plots using Matplotlib

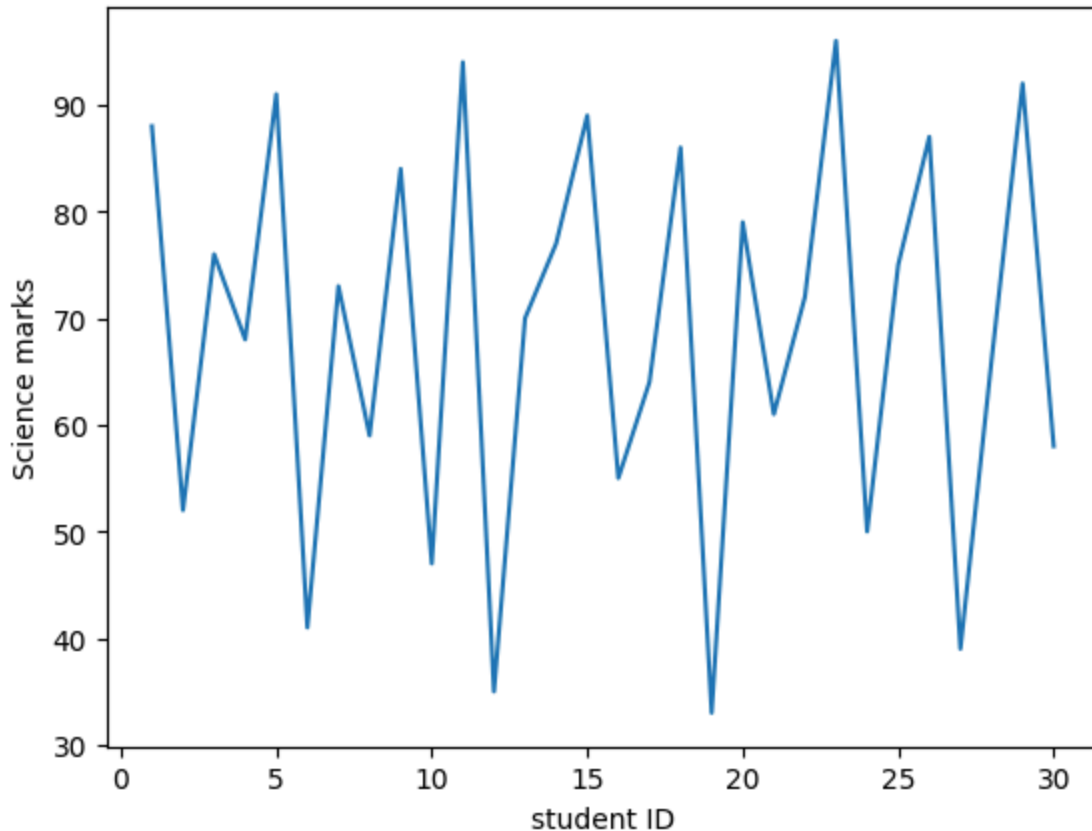
```
In [8]: plt.plot(list(df.Student_ID),list(df.Maths))  
plt.xlabel("student ID")  
plt.ylabel("math marks")  
plt.show()
```



```
In [9]: plt.plot(list(df.Student_ID),list(df.English))  
plt.xlabel("student ID")  
plt.ylabel("english marks")  
plt.show()
```



```
In [10]: plt.plot(list(df.Student_ID),list(df.Science))  
plt.xlabel("student ID")  
plt.ylabel("Science marks")  
plt.show()
```



A.3. Write a Python program for Basic plots using Matplotlib and A.7. Write a Python program to draw Scatter plots

```
In [11]: #all 3 together
# Create figure and subplots
fig, axes = plt.subplots(2, 3, figsize=(15, 6))

# Plot 1: Maths
axes[0][0].plot(df.Student_ID, df.Maths)
axes[0][0].set_title('Maths')
axes[0][0].set_xlabel('Student ID')
axes[0][0].set_ylabel('Score')

# Plot 2: English
axes[0][1].plot(df.Student_ID, df.English)
axes[0][1].set_title('English')
axes[0][1].set_xlabel('Student ID')
axes[0][1].set_ylabel('Score')

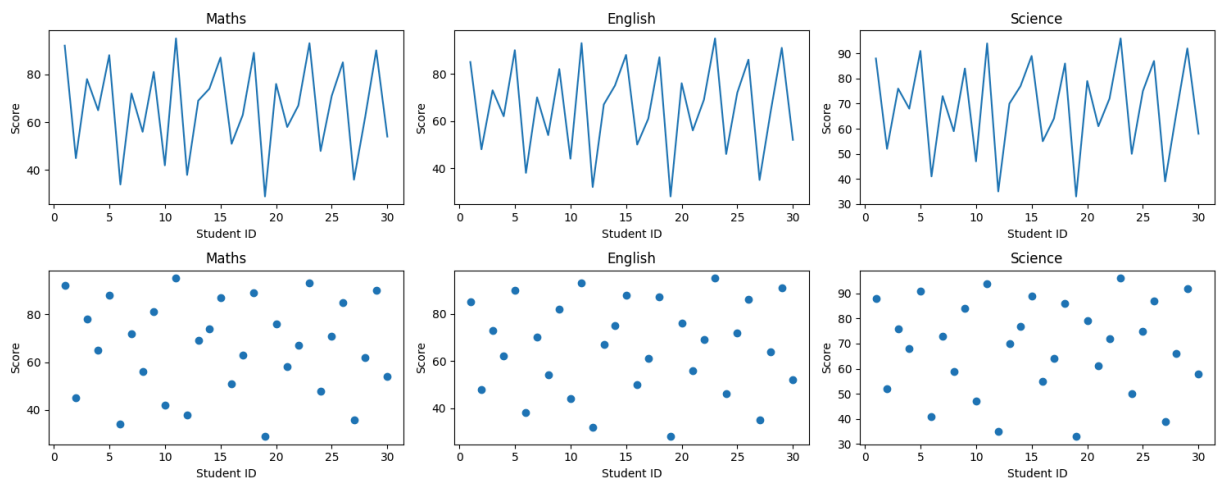
# Plot 3: Science
axes[0][2].plot(df.Student_ID, df.Science)
axes[0][2].set_title('Science')
axes[0][2].set_xlabel('Student ID')
axes[0][2].set_ylabel('Score')

axes[1][0].scatter(df.Student_ID, df.Maths)
axes[1][0].set_title('Maths')
axes[1][0].set_xlabel('Student ID')
axes[1][0].set_ylabel('Score')
```

```
# Plot 2: English
axes[1][1].scatter(df.Student_ID, df.English)
axes[1][1].set_title('English')
axes[1][1].set_xlabel('Student ID')
axes[1][1].set_ylabel('Score')

# Plot 3: Science
axes[1][2].scatter(df.Student_ID, df.Science)
axes[1][2].set_title('Science')
axes[1][2].set_xlabel('Student ID')
axes[1][2].set_ylabel('Score')

plt.tight_layout()
plt.show()
```



A.4. Write a Python program to compute Frequency distributions and A.5. Write a Python program to calculate the average and variance

A.6. Write a Python program to draw Normal Curves

The main formula for Normal Distribution is mentioned below

where mentioned formula of normal distribution is defined as the probability Density function of any random variable

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

```
In [30]: def freq(arr):
          g = (max(arr)-min(arr))/6
          fq = np.zeros(6)
          min_val = min(arr)

          for i in arr:
              if i < min_val:
                  fq[0] += 1
```



```

elif min_val <= i < min_val + g:
    fq[0] += 1 # Changed from fq[1]
elif min_val + g <= i < min_val + (2*g):
    fq[1] += 1 # Changed from fq[2]
elif min_val + (2*g) <= i < min_val + (3*g):
    fq[2] += 1 # Changed from fq[3]
elif min_val + (3*g) <= i < min_val + (4*g):
    fq[3] += 1 # Changed from fq[4]
elif min_val + (4*g) <= i < min_val + (5*g):
    fq[4] += 1 # Changed from fq[5]
else: # i >= min_val + (5*g)
    fq[5] += 1

# Create labels correctly
labels = [
    f"{min_val:.1f}-{min_val+g:.1f}",
    f"{min_val+g:.1f}-{min_val+2*g:.1f}",
    f"{min_val+2*g:.1f}-{min_val+3*g:.1f}",
    f"{min_val+3*g:.1f}-{min_val+4*g:.1f}",
    f"{min_val+4*g:.1f}-{min_val+5*g:.1f}",
    f"{min_val+5*g:.1f}-{max(arr):.1f}"
]

return [fq, labels]

def normal_dist(x, mean, sd):
    prob_density = (np.pi*sd) * np.exp(-0.5*((x-mean)/sd)**2)
    return prob_density

def nd_graph(arr, mean, sd):
    nd = []
    for i in arr:
        nd.append((np.pi*sd) * np.exp(-0.5*((i-mean)/sd)**2))
    return nd

#all 3 together
# Create figure and subplots
fig, axes = plt.subplots(2, 3, figsize=(15, 6))

math = freq(df.Maths)
# Plot 1: Maths
axes[0][0].bar(math[1], math[0])
axes[0][0].set_title('Maths')
axes[0][0].set_xlabel('score')
axes[0][0].set_ylabel('Freuency')

# Plot 2: English
english = freq(df.English)
axes[0][1].bar(english[1], english[0])
axes[0][1].set_title('English')
axes[0][1].set_xlabel('score')
axes[0][1].set_ylabel('Freuency')

# Plot 3: Science

```

```

sci = freq(df.Science)
axes[0][2].bar(sci[1], sci[0])
axes[0][2].set_title('Science')
axes[0][2].set_xlabel('score')
axes[0][2].set_ylabel('Freuency')

nd_math = nd_graph(df.Maths,np.mean(df.Maths),np.std(df.Maths))
axes[1][0].bar(df.Maths,nd_math)
axes[1][0].set_title('Maths')
axes[1][0].set_xlabel('Normal Distribution Graph')
axes[1][0].set_ylabel('probability')

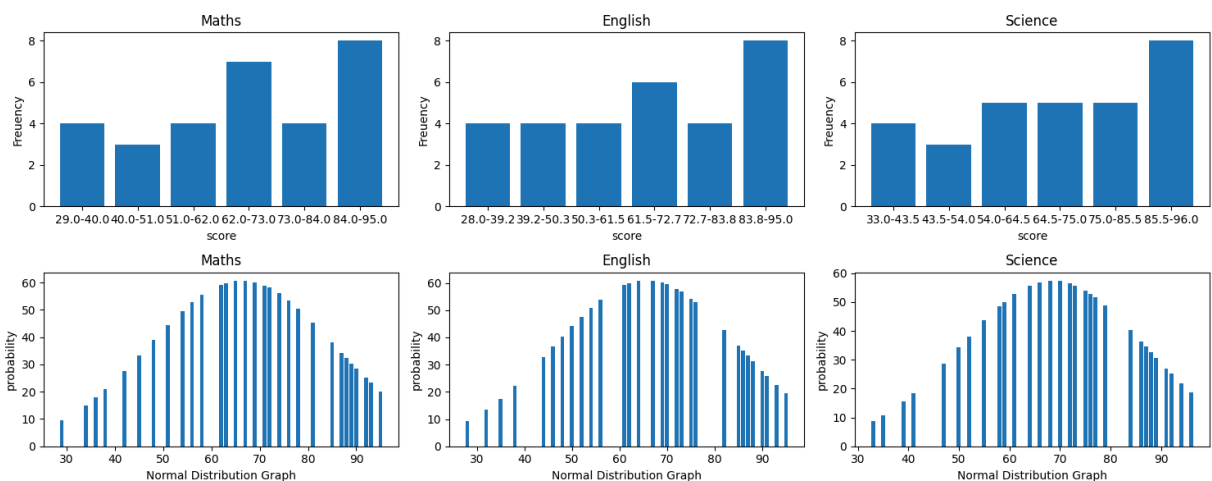
# Plot 2: English
nd_Eng = nd_graph(df.English,np.mean(df.English),np.std(df.English))
axes[1][1].bar(df.English,nd_Eng)
axes[1][1].set_title('English')
axes[1][1].set_xlabel('Normal Distribution Graph')
axes[1][1].set_ylabel('probability')

# Plot 3: Science
nd_sci = nd_graph(df.Science,np.mean(df.Science),np.std(df.Science))
axes[1][2].bar(df.Science,nd_sci)
axes[1][2].set_title('Science')
axes[1][2].set_xlabel('Normal Distribution Graph')
axes[1][2].set_ylabel('probability')

plt.tight_layout()
plt.show()

print(f"average marks of math are {np.average(list(df.Maths))} and variance is {np.var(list(df.Maths))}")
print(f"average marks of English are {np.average(list(df.English))} and variance is {np.var(list(df.English))}")
print(f"average marks of Science are {np.average(list(df.Science))} and variance is {np.var(list(df.Science))}")

```



average marks of math are 66.26666666666667 and variance is 373.86222222222216
 and standard deviation is 19.33551711804528
 average marks of English are 65.63333333333334 and variance is 376.4988888888888
 9 and standard deviation is 19.403579280351575
 average marks of Science are 68.56666666666666 and variance is 334.0455555555555
 55 and standard deviation is 18.276913184549397

A.8. Write a Python program to compute a Correlation coefficient

```
In [31]: print(f"The correlation coefficient for marks in Math and Science = \n {np.cor
```

```
The correlation coefficient for marks in Math and Science =  
[[1.          0.99176175]  
 [0.99176175 1.          ]]
```